Początkowa analiza i założenia

Założenia architektoniczne systemu zarządzania salonem masażu – Afrodyta

1. Wprowadzenie i cel projektu

Przedmiotem projektu jest opracowanie aplikacji informatycznej przeznaczonej do zarządzania działalnością salonu masażu. System ma wspierać organizację codziennej pracy salonu, ułatwiać zarządzanie jego zasobami, oraz umożliwiać sprawną obsługę informacji związanych z działalnością placówki.

Projektowana aplikacja będzie służyć do gromadzenia, przetwarzania i udostępniania danych niezbędnych do funkcjonowania salonu. Wśród obszarów, które system będzie obejmował, znajdą się między innymi informacje o klientach, pracownikach, oferowanych usługach oraz rezerwacjach. Szczegółowy zakres funkcjonalny zostanie określony na dalszym etapie analizy wymagań.

Jednym z pierwszych założeń projektowych było wybranie odpowiedniego rodzaju aplikacji i zdecydowano się na rozwiązanie internetowe, działające w architekturze **klient–serwer**.

Oznacza to, że użytkownik będzie korzystał z systemu za pośrednictwem przeglądarki internetowej, bez konieczności instalowania dedykowanego programu na swoim urządzeniu. Interfejs aplikacji będzie prezentowany w postaci strony internetowej, natomiast przetwarzanie danych i realizacja logiki biznesowej będą odbywały się po stronie serwera.

Wybór technologii webowej zapewnia możliwość korzystania z aplikacji za pomocą różnych urządzeń wyposażonych w przeglądarkę internetową. Rozwiązanie takie ułatwia również centralne zarządzanie systemem, jego aktualizację oraz utrzymywanie danych w jednym miejscu.

2. Przyjęta architektura systemu

Na podstawie wstępnej analizy przyjęto, że aplikacja zostanie zbudowana z wykorzystaniem architektury trójwarstwowej:

- **Three-Tier Architecture**

Jest to model organizacji systemu informatycznego, w którym poszczególne zadania są rozdzielone pomiędzy trzy współpracujące ze sobą warstwy:

1. **Warstwę prezentacji – Presentation Layer**
2. **Warstwę logiki aplikacji – Application Layer / Business Logic Layer**
3. **Warstwę danych – Data Layer**

Każda z warstw będzie pełniła odrębną funkcję, a ich współpraca umożliwi realizację funkcjonalności całej aplikacji. Przy tak przyjętym modelu architektury pojawiają się również dwie warstwy komunikacji wymagające implementacji. Ich założenia też będą opisane w tym dokumencie.

Takie rozdzielenie odpowiedzialności pozwala zachować uporządkowaną strukturę projektu, ułatwia rozwijanie poszczególnych elementów systemu oraz ogranicza wzajemne uzależnienie warstwy interfejsu, logiki biznesowej i przechowywania danych.

3. Charakterystyka poszczególnych warstw aplikacji

3.1. Warstwa prezentacji – Presentation Layer

Warstwa prezentacji będzie bezpośrednim interfejsem komunikacji pomiędzy użytkownikiem a systemem.

Zostanie zrealizowana w postaci strony internetowej wyświetlanej w przeglądarce użytkownika. Jej zadaniem będzie prezentowanie informacji, udostępnianie funkcji aplikacji oraz umożliwienie wprowadzania i modyfikowania danych.

Do realizacji tej warstwy zostaną wykorzystane następujące technologie:

1. **HTML – język znaczników służący do budowania struktury dokumentów i elementów interfejsu, takich jak formularze, tabele, przyciski czy sekcje strony.**
2. **CSS – język arkuszy stylów odpowiedzialny za wygląd aplikacji, rozmieszczenie elementów, kolorystykę i dostosowanie interfejsu do różnych rozmiarów ekranów.**
3. **JavaScript – język programowania wykorzystywany do obsługi interakcji użytkownika, dynamicznej aktualizacji widoków oraz komunikacji z serwerem.**

Warstwa ta będzie odpowiadała przede wszystkim za sposób prezentowania informacji i obsługę działań wykonywanych przez użytkownika. Nie będzie bezpośrednio

łączyła się z bazą danych. Wszelkie operacje wymagające dostępu do danych będą realizowane za pośrednictwem warstwy logiki aplikacji, udostępnianej przez serwer WWW pod określonym adresem URL.

3.2. Warstwa logiki aplikacji – Application Layer

Warstwa logiki aplikacji będzie centralnym elementem odpowiedzialnym za przetwarzanie żądań użytkowników oraz komunikację z magazynem danych, czyli bazą danych MySQL.

Zostanie zaimplementowana w języku **PHP**, działającym po stronie serwera.

Do jej podstawowych zadań będą należały:

1. **odbieranie i przetwarzanie żądań przesyłanych z przeglądarki internetowej;**
2. **walidacja danych wprowadzanych przez użytkowników;**
3. **realizacja reguł biznesowych określających sposób działania aplikacji;**
4. **kontrolowanie dostępu do poszczególnych funkcjonalności;**
5. **przygotowywanie odpowiedzi przekazywanych do warstwy prezentacji;**
6. **komunikacja z bazą danych i wykonywanie operacji na przechowywanych informacjach.**

Warstwa ta będzie pośredniczyła pomiędzy interfejsem użytkownika a bazą danych. Dzięki temu logika działania systemu pozostanie oddzielona zarówno od sposobu prezentowania informacji, jak i od fizycznego sposobu ich przechowywania.

3.3. Warstwa danych – Data Layer

Warstwa danych będzie odpowiadała za trwałe przechowywanie informacji wykorzystywanych przez aplikację.

Jako system zarządzania bazą danych zostanie wykorzystany **MySQL**. Na etapie przygotowania środowiska dopuszczalne będzie również wykorzystanie MariaDB jako kompatybilnego rozwiązania z tej samej rodziny systemów bazodanowych.

Baza danych będzie pełniła funkcję centralnego magazynu informacji dotyczących działalności salonu masażu.

Wśród przewidywanych kategorii danych znajdą się między innymi:

- dane klientów;
- dane pracowników;

- informacje o usługach i metodach masażu;
- informacje o dostępnych zasobach salonu;
- rezerwacje i terminy wizyt;
- inne informacje wynikające z przyjętego zakresu funkcjonalnego systemu.

Do komunikacji pomiędzy aplikacją PHP a bazą danych zostanie wykorzystany język SQL. Dostęp do bazy danych będzie realizowany z poziomu PHP, z wykorzystaniem interfejsu **PDO – PHP Data Objects**.

Warstwa danych będzie odpowiadała za zapisywanie, odczytywanie, aktualizowanie i usuwanie informacji. Jej zadaniem będzie również zapewnienie odpowiedniej struktury danych, powiązań pomiędzy poszczególnymi elementami bazy oraz stabilności.

4. Komunikacja pomiędzy warstwami

W przyjętej architekturze trójwarstwowej poszczególne warstwy systemu współpracują ze sobą za pośrednictwem dwóch niezależnych kanałów komunikacji. Każdy z nich wykorzystuje inne technologie i mechanizmy, dostosowane do rodzaju przesyłanych informacji oraz zadań realizowanych przez współpracujące elementy systemu.

1. **Pierwszy kanał odpowiada za komunikację pomiędzy warstwą prezentacji, działającą w przeglądarce internetowej, a warstwą logiki aplikacji PHP, uruchomioną na serwerze WWW.**

2. **Drugi kanał zapewnia komunikację pomiędzy warstwą logiki aplikacji PHP a warstwą danych, której podstawę stanowi system zarządzania bazą danych MySQL.**

4.1. Komunikacja pomiędzy przeglądarką internetową a serwerem PHP

Komunikacja pomiędzy przeglądarką internetową a serwerem aplikacji PHP będzie odbywała się z wykorzystaniem protokołu HTTPS.

Żądania będą kierowane do dedykowanych punktów dostępu – **endpointów**, obsługiwanych przez aplikację PHP działającą po stronie serwera. Poszczególne endpointy będą odpowiadały za przyjmowanie określonych rodzajów żądań i uruchamianie właściwej logiki aplikacji.

Wymiana danych pomiędzy tymi elementami będzie realizowana w formacie **JSON – JavaScript Object Notation**. Jest to tekstowy format zapisu i wymiany uporządkowanych danych, umożliwiający ich przekazywanie pomiędzy różnymi elementami systemu.

Po stronie przeglądarki za inicjowanie żądań i obsługę otrzymywanych odpowiedzi będzie odpowiadał język JavaScript, a w szczególności jego asynchroniczna metoda **Fetch API**. Zastosowanie tego mechanizmu pozwoli uniknąć konieczności przeładowywania całej strony podczas wysyłania żądań przez użytkownika. Umożliwi również zachowanie wcześniej pobranych danych oraz wyników operacji wykonywanych przez użytkownika, bez konieczności ponownego pobierania wszystkich informacji z serwera.

Po stronie serwera PHP będzie odbierał żądania, przetwarzał przekazane informacje i przygotowywał odpowiedzi w postaci danych zakodowanych w formacie JSON.

Takie rozwiązanie pozwoli oddzielić sposób prezentowania informacji użytkownikowi od mechanizmów odpowiedzialnych za przetwarzanie danych i realizację logiki biznesowej.

4.2. Komunikacja pomiędzy serwerem PHP a bazą danych MySQL

Drugi kanał komunikacji będzie wykorzystywany do wymiany informacji pomiędzy warstwą logiki aplikacji a warstwą danych.

Połączenie z bazą danych MySQL oraz wykonywanie operacji na przechowywanych informacjach będą realizowane przez aplikację PHP z wykorzystaniem interfejsu **PDO – PHP Data Objects**.

PDO stanowi interfejs programistyczny PHP umożliwiający nawiązywanie połączeń z bazami danych oraz wykonywanie na nich operacji. Do formułowania zapytań i poleceń kierowanych do bazy danych będzie wykorzystywany język **SQL – Structured Query Language**.

Za pomocą zapytań SQL aplikacja będzie mogła między innymi odczytywać, dodawać, modyfikować i usuwać informacje przechowywane w bazie danych.

Wyniki wykonanych operacji będą przekazywane z powrotem do aplikacji PHP, gdzie zostaną odpowiednio przetworzone i wykorzystane do przygotowania odpowiedzi dla przeglądarki internetowej.

Zastosowanie PDO pozwoli oddzielić logikę aplikacji od szczegółów nawiązywania połączenia z bazą danych i wykonywania operacji bazodanowych.

4.3. Podsumowanie mechanizmów komunikacji

Przyjęte rozwiązanie zakłada wykorzystanie dwóch odrębnych mechanizmów komunikacji:

1. **Przeglądarka internetowa – serwer PHP**: protokół HTTP/HTTPS jako mechanizm przesyłania żądań i odpowiedzi, format JSON jako sposób reprezentacji wymienianych danych oraz JavaScript i PHP jako technologie obsługujące komunikację po obu stronach.

2. **Serwer PHP – baza danych MySQL**: interfejs PDO wykorzystywany przez PHP do nawiązywania połączenia i wykonywania operacji oraz język SQL służący do formułowania zapytań bazodanowych.

Rozdzielenie tych mechanizmów pozwala zachować niezależność komunikacji z użytkownikiem od operacji wykonywanych na bazie danych. Warstwa logiki aplikacji PHP pełni w tym modelu funkcję pośredniczącą, kontrolując przepływ informacji pomiędzy interfejsem użytkownika a magazynem danych.

Przykładowy przebieg operacji w systemie:

1. Użytkownik wykonuje określoną czynność w interfejsie aplikacji, np. próbuje wyświetlić listę rezerwacji.
2. Przeglądarka, za pośrednictwem kodu JavaScript, wysyła odpowiednie żądanie HTTP/HTTPS do dedykowanego punktu dostępu (endpointu) aplikacji działającej na serwerze WWW.
3. Serwer odbiera żądanie i przekazuje je do odpowiedniego mechanizmu obsługi, który uruchamia właściwą logikę aplikacji.
4. Jeżeli operacja wymaga dostępu do danych, PHP komunikuje się z bazą MySQL za pomocą zapytań SQL, wykorzystując interfejs PDO.
5. Baza danych zwraca wymagane informacje.
6. PHP przetwarza otrzymane dane, przygotowuje odpowiedź w formacie JSON i wysyła ją do przeglądarki.
7. Przeglądarka odbiera odpowiedź za pośrednictwem protokołu HTTP/HTTPS. Następnie JavaScript dekoduje otrzymane dane, przetwarza je i aktualizuje interfejs strony WWW, prezentując wynik użytkownikowi.

5. Uzasadnienie przyjętego rozwiązania

Wybór architektury trójwarstwowej oraz technologii webowych wynika z potrzeby stworzenia systemu, który będzie czytelny pod względem konstrukcji, możliwy do dalszego rozwoju i łatwy w utrzymaniu.

Najważniejsze zalety przyjętego rozwiązania to:

- **Rozdzielenie odpowiedzialności** – każda warstwa realizuje określoną grupę zadań, co ułatwia organizację kodu i prac zespołu.

- **Możliwość niezależnego rozwijania elementów** – zmiany w wyglądzie interfejsu nie muszą wymagać modyfikacji logiki biznesowej lub struktury bazy danych.
- **Centralne zarządzanie danymi** – informacje są przechowywane w jednym miejscu, a dostęp do nich odbywa się za pośrednictwem serwera aplikacji.
- **Dostęp przez przeglądarkę internetową** – użytkownik może korzystać z systemu bez instalowania dedykowanego programu.
- **Możliwość rozbudowy** – przyjęta struktura ułatwia dodawanie kolejnych funkcjonalności w miarę rozwoju projektu.
- **Czytelny podział zadań** – rozdzielenie warstw pozwala wyodrębnić obszary odpowiedzialności podczas planowania i realizacji zadań zespołowych.

6. Założenia do dalszych prac projektowych

Przedstawiona architektura stanowi wstępną koncepcję techniczną systemu. Na jej podstawie będą prowadzone dalsze prace związane z analizą wymagań, projektowaniem i implementacją aplikacji.

W kolejnych etapach projektu przewiduje się między innymi:

- Szczegółowe określenie funkcjonalności systemu i potrzeb jego użytkowników.
- Opracowanie struktury interfejsu internetowego oraz podstawowych widoków aplikacji.
- Zaprojektowanie struktury backendu PHP i podziału logiki aplikacji na odpowiednie moduły.
- Opracowanie modelu danych, tabel, relacji i ograniczeń w bazie MySQL.
- Określenie sposobu komunikacji pomiędzy poszczególnymi elementami systemu.
- Przygotowanie środowiska programistycznego i testowego.
- Implementację, integrację i testowanie poszczególnych funkcjonalności.

Szczegółowe decyzje dotyczące struktury poszczególnych warstw, organizacji kodu, modelu danych i interfejsów komunikacyjnych będą dokumentowane w miarę postępu prac.

7. Podsumowanie

Przyjęto koncepcję internetowej aplikacji do zarządzania salonem masażu, opartej na architekturze trójwarstwowej – **Three-Tier Architecture**.

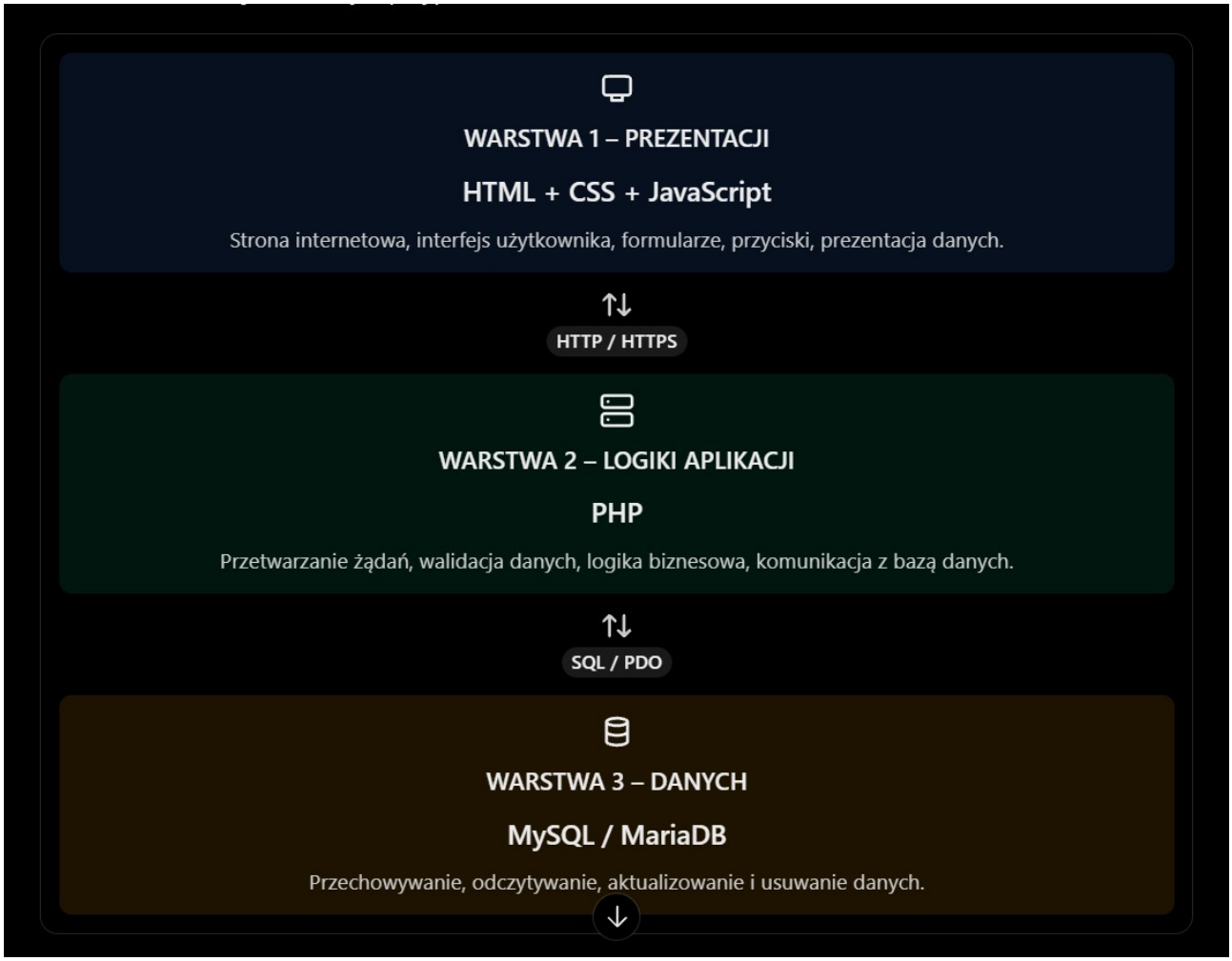
Warstwa prezentacji będzie realizowana w przeglądarce internetowej z wykorzystaniem HTML, CSS i JavaScript. Warstwa logiki aplikacji zostanie zaimplementowana w PHP, natomiast za trwałe przechowywanie informacji będzie odpowiadała baza danych MySQL.

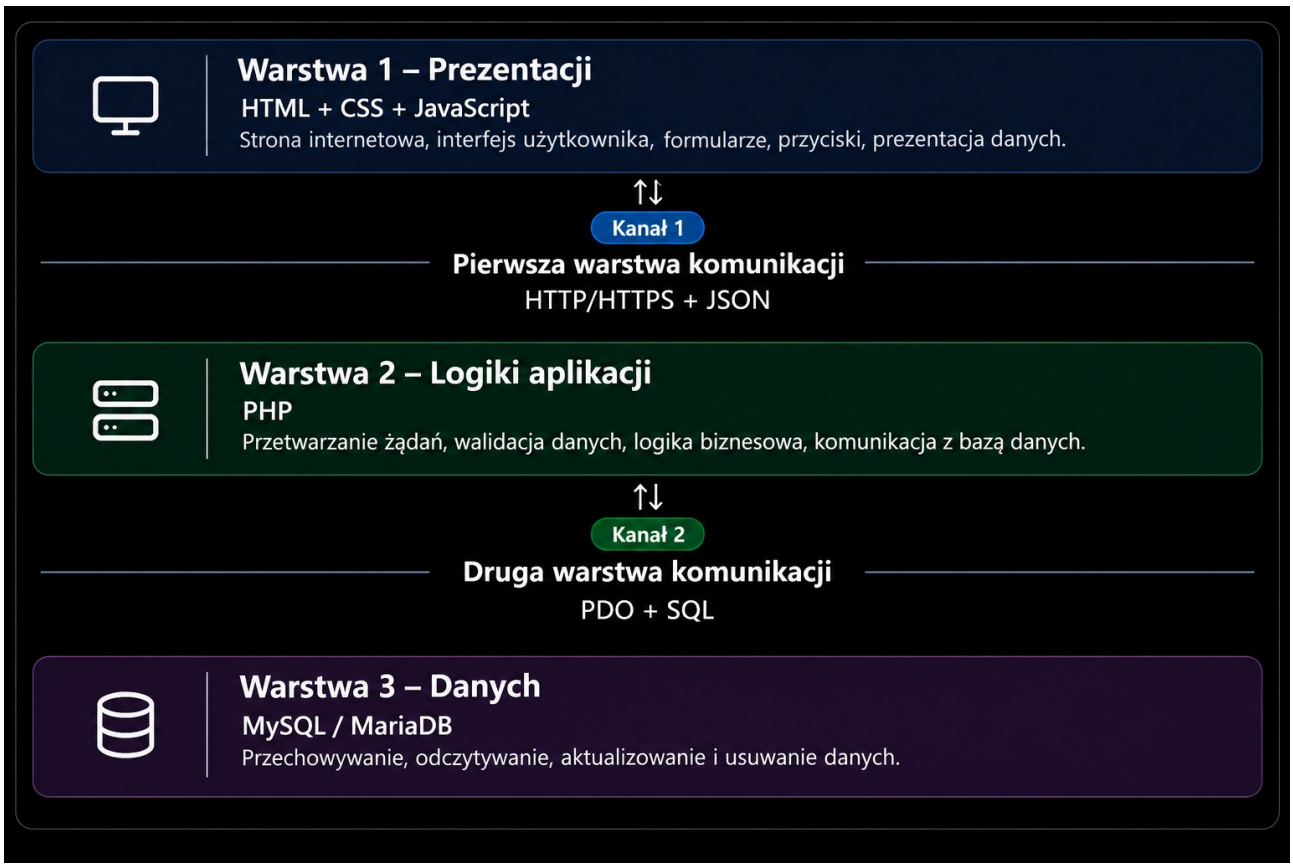
Komunikacja pomiędzy przeglądarką internetową a serwerem będzie odbywała się za pośrednictwem protokołu HTTP/HTTPS, z wykorzystaniem dedykowanych punktów dostępu – endpointów obsługiwanych przez aplikację PHP. Wymiana danych pomiędzy tymi warstwami będzie realizowana w formacie JSON.

Logika aplikacji PHP, obsługująca poszczególne endpointy, będzie odpowiadała za komunikację z bazą danych z wykorzystaniem zapytań SQL oraz interfejsu PDO.

Przyjęta koncepcja stanowi punkt wyjścia do dalszego projektowania systemu oraz podziału prac pomiędzy członków zespołu projektowego.

8. Schemat graficzny przepływu komunikacji:




Warstwa 1 – Prezentacji
HTML + CSS + JavaScript
Strona internetowa, interfejs użytkownika, formularze, przyciski, prezentacja danych.
Kanał 1
Pierwsza warstwa komunikacji
HTTP/HTTPS + JSON
Warstwa 2 – Logiki aplikacji
PHP
Przetwarzanie żądań, walidacja danych, logika biznesowa, komunikacja z bazą danych.
Kanał 2
Druga warstwa komunikacji
PDO + SQL
Warstwa 3 – Danych
MySQL / MariaDB
Przechowywanie, odczytywanie, aktualizowanie i usuwanie danych.